

Design and Analysis of Algorithms

Greedy: Huffman Coding

A file must cross a wire that carries only 0 and 1.
How few bits are enough?

Dr. Golzari

University of Hormozgan

Fall 1405

Where We're Going

1. The problem: files are just bits
2. Plan A: same-length codes
3. Plan B: short codes for common letters
4. The trouble → prefix codes
5. The algorithm → walkthrough
6. Encode, decode, and count the savings

The Problem

What's the Problem?

- Files are huge; wires and disks are finite.

What's the Problem?

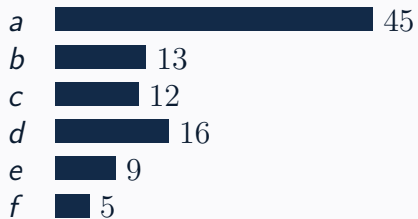
- Files are huge; wires and disks are finite.
- Text today: one byte (8 bits) per character — always, regardless of the character.

Today's goal

Send the **same file** in **fewer bits**.

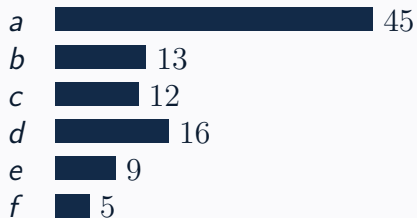
Our Test File

100 characters, six different letters:



Our Test File

100 characters, six different letters:



Notice: *a* shows up $9\times$ more than *f*.

Plan A: Same Length for Everyone

Six letters need $\lceil \log_2 6 \rceil = 3$ bits each:

$$a = 000, \quad b = 001, \quad \dots, \quad f = 101$$

Plan A: Same Length for Everyone

Six letters need $\lceil \log_2 6 \rceil = 3$ bits each:

$$a = 000, \quad b = 001, \quad \dots, \quad f = 101$$

$$100 \text{ chars} \times 3 \text{ bits} = 300 \text{ bits}$$

Plan A: Same Length for Everyone

Six letters need $\lceil \log_2 6 \rceil = 3$ bits each:

$$a = 000, \quad b = 001, \quad \dots, \quad f = 101$$

$$100 \text{ chars} \times 3 \text{ bits} = 300 \text{ bits}$$

- It works — but a pays the same as f .

Plan A: Same Length for Everyone

Six letters need $\lceil \log_2 6 \rceil = 3$ bits each:

$$a = 000, \quad b = 001, \quad \dots, \quad f = 101$$

$$100 \text{ chars} \times 3 \text{ bits} = 300 \text{ bits}$$

- It works — but a pays the same as f .
- **Can we do better?**

Plan B: Charge by Popularity

- Common letter → **short** code.
- Rare letter → **long** code.

Plan B: Charge by Popularity

- Common letter → **short** code.
- Rare letter → **long** code.

Morse did it in 1838:

E = . T = - Q = --.-

Plan B: Charge by Popularity

- Common letter → **short** code.
- Rare letter → **long** code.

Morse did it in 1838:

E = . T = - Q = --.-

Sounds great. What could go wrong?

The Trouble

Try: $a = 0$, $b = 01$, $c = 1$ A friend sends you: 01

The Trouble

Try: $a = 0$, $b = 01$, $c = 1$ A friend sends you: 01
Is that ...

$\underbrace{01}_{a, c}$ or $\underbrace{01}_b ?$

The Trouble

Try: $a = 0$, $b = 01$, $c = 1$ A friend sends you: 01

Is that ...

$\underbrace{01}_{a, c}$ or $\underbrace{01}_b ?$

Ambiguous!

We don't know where one code ends and the next begins.

Which of these codes is **unambiguous**?

(1) $a=0, b=00, c=1$ (2) $a=0, b=10, c=11$ (3) $a=00, b=01, c=0$

Which of these codes is **unambiguous**?

(1) $a=0, b=00, c=1$ (2) $a=0, b=10, c=11$ (3) $a=00, b=01, c=0$

Answer: (2). No code is the beginning of another.

Prefix Codes to the Rescue

The rule

No code is the prefix of **another** code.

The rule

No code is the prefix of **another** code.

- Read bits left to right.
- First complete match wins — output the letter, keep reading.

The rule

No code is the prefix of **another** code.

- Read bits left to right.
- First complete match wins — output the letter, keep reading.
- No separators, no guessing.

Prefix Codes to the Rescue

The rule

No code is the prefix of **another** code.

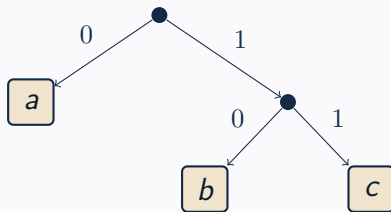
- Read bits left to right.
- First complete match wins — output the letter, keep reading.
- No separators, no guessing.

One rule, one reading

Prefix-free $\Rightarrow$ decoding is **always** unique.

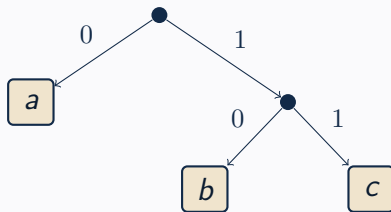
Trees Make It Visual

Left edge = 0, right edge = 1. A code = the path to a **leaf**.



Trees Make It Visual

Left edge = 0, right edge = 1. A code = the path to a **leaf**.



Codes: $a=0$, $b=10$, $c=11$ — prefix-free **by construction**.

Which Tree Wins?

A letter sitting at depth d costs $\text{freq} \times d$ bits.

$$\text{total} = \sum \text{freq} \times \text{depth}$$

Which Tree Wins?

A letter sitting at depth d costs $\text{freq} \times d$ bits.

$$\text{total} = \sum \text{freq} \times \text{depth}$$

- So the game is now: **build the cheapest tree.**

Which Tree Wins?

A letter sitting at depth d costs $\text{freq} \times d$ bits.

$$\text{total} = \sum \text{freq} \times \text{depth}$$

- So the game is now: **build the cheapest tree.**
- Common letters: shallow. Rare letters: deep.

The Algorithm

Huffman's Big Insight

Think about the deepest leaves.

Huffman's Big Insight

Think about the deepest leaves.

- A *rare* letter down there: cheap.
- A *common* letter down there: disaster.

Think about the deepest leaves.

- A *rare* letter down there: cheap.
- A *common* letter down there: disaster.
- So the **two rarest** letters belong deepest — as **siblings**.

Huffman's Big Insight

Think about the deepest leaves.

- A *rare* letter down there: cheap.
- A *common* letter down there: disaster.
- So the **two rarest** letters belong deepest — as **siblings**.

The greedy rule

Merge the two smallest weights; repeat.

The Algorithm

1. Put every letter in a pool, by weight.

The Algorithm

1. Put every letter in a pool, by weight.
2. Pull out the **two smallest**; merge them into one node with their **summed** weight; toss it back in.

The Algorithm

1. Put every letter in a pool, by weight.
2. Pull out the **two smallest**; merge them into one node with their **summed** weight; toss it back in.
3. Repeat until one node remains. That's your tree.

The Algorithm

1. Put every letter in a pool, by weight.
2. Pull out the **two smallest**; merge them into one node with their **summed** weight; toss it back in.
3. Repeat until one node remains. That's your tree.

n letters $\Rightarrow$ exactly $n - 1$ merges.

Walkthrough: The Pool

a:45 b:13 c:12 d:16 e:9 f:5

Walkthrough: The Pool

a:45 b:13 c:12 d:16 e:9 f:5

Sorted pool:

5	9	12	13	16	45
---	---	----	----	----	----

Walkthrough: The Pool

a:45 b:13 c:12 d:16 e:9 f:5

Sorted pool:

5 9 12 13 16 45

- Grab the two smallest ... and merge.

Walkthrough: The Merges

- $5 + 9 \rightarrow 14$: 12, 13, 14, 16, 45

Walkthrough: The Merges

- $5 + 9 \rightarrow 14$: 12, 13, 14, 16, 45
- $12 + 13 \rightarrow 25$: 14, 16, 25, 45

Walkthrough: The Merges

- $5 + 9 \rightarrow 14$: 12, 13, 14, 16, 45
- $12 + 13 \rightarrow 25$: 14, 16, 25, 45
- $14 + 16 \rightarrow 30$: 25, 30, 45

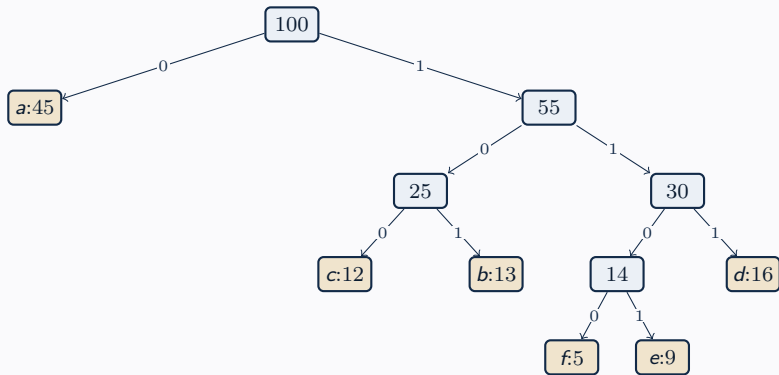
Walkthrough: The Merges

- $5 + 9 \rightarrow 14$: 12, 13, 14, 16, 45
- $12 + 13 \rightarrow 25$: 14, 16, 25, 45
- $14 + 16 \rightarrow 30$: 25, 30, 45
- $25 + 30 \rightarrow 55$: 45, 55

Walkthrough: The Merges

- $5 + 9 \rightarrow 14$: 12, 13, 14, 16, 45
- $12 + 13 \rightarrow 25$: 14, 16, 25, 45
- $14 + 16 \rightarrow 30$: 25, 30, 45
- $25 + 30 \rightarrow 55$: 45, 55
- $45 + 55 \rightarrow 100$ — done!

The Tree We Built



Walk root $\rightarrow$ leaf; collect the 0s and 1s:

letter	code	length
<i>a</i>	0	1
<i>c</i>	100	3
<i>b</i>	101	3
<i>d</i>	111	3
<i>f</i>	1100	4
<i>e</i>	1101	4

Walk root $\rightarrow$ leaf; collect the 0s and 1s:

letter	code	length
<i>a</i>	0	1
<i>c</i>	100	3
<i>b</i>	101	3
<i>d</i>	111	3
<i>f</i>	1100	4
<i>e</i>	1101	4

Just as planned: *a* is **1** bit; *f*, *e* pay **4**.

How'd We Do?

letter	freq	depth	bits
<i>a</i>	45	1	45
<i>b</i>	13	3	39
<i>c</i>	12	3	36
<i>d</i>	16	3	48
<i>e</i>	9	4	36
<i>f</i>	5	4	20

How'd We Do?

letter	freq	depth	bits
<i>a</i>	45	1	45
<i>b</i>	13	3	39
<i>c</i>	12	3	36
<i>d</i>	16	3	48
<i>e</i>	9	4	36
<i>f</i>	5	4	20

300 bits $\longrightarrow$ 224 bits **25% smaller.**

How'd We Do?

letter	freq	depth	bits
<i>a</i>	45	1	45
<i>b</i>	13	3	39
<i>c</i>	12	3	36
<i>d</i>	16	3	48
<i>e</i>	9	4	36
<i>f</i>	5	4	20

300 bits $\longrightarrow$ 224 bits **25% smaller.**

Hand-check: $14 + 25 + 30 + 55 + 100 = 224$ (the merge sums!).

Try It: Encode

Encode ababa:

Try It: Encode

Encode ababa:

$$\begin{array}{ccccc} 0 & 101 & 0 & 101 & 0 \\ \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} & \underbrace{\hspace{1cm}} \\ a & b & a & b & a \end{array}$$

Try It: Encode

Encode ababa:

$$\underbrace{0}_a \underbrace{101}_b \underbrace{0}_a \underbrace{101}_b \underbrace{0}_a$$

$$= 010101010 \quad \text{— 9 bits} \quad (\text{fixed-length: 15})$$

Try It: Encode

Encode ababa:

$$\underbrace{0}_a \underbrace{101}_b \underbrace{0}_a \underbrace{101}_b \underbrace{0}_a$$

$$= 010101010 \quad \text{— 9 bits} \quad (\text{fixed-length: 15})$$

No separators needed. The receiver just reads.

Try It: Decode

Received: 010101010

Try It: Decode

Received: 010101010

Walk the tree from the root, bit by bit; leaf? Output, restart:

Try It: Decode

Received: 010101010

Walk the tree from the root, bit by bit; leaf? Output, restart:

$0 \rightarrow a$ $101 \rightarrow b$ $0 \rightarrow a$ $101 \rightarrow b$ $0 \rightarrow a$

Try It: Decode

Received: 010101010

Walk the tree from the root, bit by bit; leaf? Output, restart:

$0 \rightarrow a$ $101 \rightarrow b$ $0 \rightarrow a$ $101 \rightarrow b$ $0 \rightarrow a$

ababa — exactly what was sent. ✓

Why Is Greedy Safe Here?

1. Take any optimal tree — it has a **deepest pair of siblings**.

Why Is Greedy Safe Here?

1. Take any optimal tree — it has a **deepest pair of siblings**.
2. Swap the **two rarest** letters into those seats: smallest weights at the biggest depths $\Rightarrow$ cost can't go up.

Why Is Greedy Safe Here?

1. Take any optimal tree — it has a **deepest pair of siblings**.
2. Swap the **two rarest** letters into those seats: smallest weights at the biggest depths $\Rightarrow$ cost can't go up.
3. Glue them together (weight $x + y$) and recurse on the smaller problem.

Why Is Greedy Safe Here?

1. Take any optimal tree — it has a **deepest pair of siblings**.
2. Swap the **two rarest** letters into those seats: smallest weights at the biggest depths $\Rightarrow$ cost can't go up.
3. Glue them together (weight $x + y$) and recurse on the smaller problem.

Conclusion

Huffman's tree is **optimal** among all prefix codes.

How Fast?

$n - 1$ merges, each needing two extract-min + one insert.

How Fast?

$n - 1$ merges, each needing two extract-min + one insert.

- **Priority queue (min-heap):** $O(n \log n)$ total.
- Frequencies already sorted? Two queues: $O(n)$.

Wrap-Up

- Fixed-length: simple, wasteful.

Summary

- Fixed-length: simple, wasteful.
- Variable-length needs **prefix-free** — otherwise decoding is ambiguous.

Summary

- Fixed-length: simple, wasteful.
- Variable-length needs **prefix-free** — otherwise decoding is ambiguous.
- Prefix codes = **leaves of a binary tree**.

- Fixed-length: simple, wasteful.
- Variable-length needs **prefix-free** — otherwise decoding is ambiguous.
- Prefix codes = **leaves of a binary tree**.
- **Huffman**: merge the two rarest, $n - 1$ times — provably optimal, $O(n \log n)$.

Summary

- Fixed-length: simple, wasteful.
- Variable-length needs **prefix-free** — otherwise decoding is ambiguous.
- Prefix codes = **leaves of a binary tree**.
- **Huffman**: merge the two rarest, $n - 1$ times — provably optimal, $O(n \log n)$.
- Our file: **300** → **224 bits**.

Next session

More greedy classics: minimum spanning trees.

1. Huffman for $A:8$, $B:3$, $C:2$, $D:2$: build the tree, write the codes, compare with 2-bit fixed.
2. Decode 10010111001101 with today's codebook.
3. Encode fade, then decode it back.
4. When does Huffman save *nothing*? Why?
5. **(Challenge)** Show total bits = sum of all merge results — always.